DevCatalyst Website: Technical Implementation Document

The DevCatalyst site is a five-page public website plus a password-protected admin panel, shipped as one Vercel project: a Vite + React frontend, a FastAPI backend on Vercel serverless, Neon Postgres for data and Cloudinary for photos. Everything runs on free tiers.

Overview and goals

Goals

- Public site with Home, About, Events, Gallery and Contact pages, smooth animations, and first-load JavaScript of 120 KB gzipped or less on public pages.
- Admin panel where the team creates events, builds a registration form per event, reads and exports registrations, uploads gallery photos and reads contact messages.
- Contact submissions are saved to the database, appended to a Google Sheet and emailed to the team.
- Zero monthly cost, with a clear upgrade path if traffic grows.

Pages and their data

Page	Purpose	Data source
Home	Hero, highlights, upcoming events, call to action	Static content + next events from API
About	Mission, team, timeline	Static content (edited in code)
Events	Upcoming and past events, registration	Events and forms tables
Gallery	One album per event, full-screen viewer	Events and photos tables, images on Cloudinary
Contact	Message form, social links	Contact messages table, Sheet and email
Admin (/admin)	Manage everything above	Authenticated API

Out of scope for version 1: payments, participant accounts, file-upload fields inside registration forms (planned for phase 2) and a visual editor for the About page.
Constraint to confirm: Vercel's Hobby (free) plan is meant for non-commercial

use. A student or community club normally fits, but check the current terms before launch.

Tech stack decision

Use **Vite + React + TypeScript** for the frontend. It ships a small static bundle, has the richest animation ecosystem, suits an interactive admin panel, and needs no server runtime because FastAPI already owns the backend logic.
Frontend options compared (bundle sizes are approximate, measure with a bundle analyzer once the app exists)

Option	Baseline JS, gzipped	Animation ecosystem	Fit with a separate FastAPI backend	Verdict
Vite + React (static build, prerendered public pages)	about 55 to 70 KB	Largest: Motion, GSAP, Lenis, all React-friendly	Plain JSON over /api, typed client from OpenAPI	**Chosen**
Next.js	about 90 to 110 KB	Same as React	Its server features duplicate FastAPI, and it adds build and runtime weight	Rejected: heavier, no benefit here
Astro + React islands	Near 0 KB for static sections	Good, but animation code lives in islands	Fine for public pages, awkward for a large interactive admin app	Strong runner-up for a content-only site
SvelteKit	about 20 to 30 KB	Built-in transitions, fewer drag-and-drop and form libraries	Fine	Rejected: smaller ecosystem for the admin form builder

Chosen stack

Layer	Choice	Why
Build and language	Vite, React, TypeScript	Fast builds, small output, type safety end to end
Routing	React Router with lazy-loaded routes	Admin code never loads for visitors
Styling	Tailwind CSS	Unused styles are stripped, so CSS stays around 10 KB
Animation	CSS, Motion (Framer Motion) via LazyMotion, GSAP + ScrollTrigger on Home only, Lenis for smooth scroll	See the table below
Server data	TanStack Query + client generated from FastAPI's OpenAPI schema (openapi-typescript)	Caching, retries and types that match the backend
Forms	React Hook Form + Zod	The public form renderer builds its Zod schema from the stored form definition
Admin extras	dnd-kit for reordering form fields	Lives only in the admin chunk
Prerendering	vite-react-ssg for Home, About, Events, Gallery, Contact	Fast first paint and crawlable HTML
Fonts	Self-hosted variable fonts (Fontsource), one or two families	No third-party font requests
Backend	FastAPI, Pydantic v2, SQLAlchemy 2 (async), Alembic	Matches your plan, with automatic OpenAPI docs
Database	Neon Postgres	Free tier does not pause for a week of idleness the way Supabase's does

Photos	Cloudinary	Resizing and modern formats on the fly, so we never store thumbnails
Email	Apps Script MailApp, run from the Sheet	No mail password in the API, and alerts keep working if the API is down
Sheets	Google Sheets API with a service account	No user login needed for appends

Animation toolkit and its cost (sizes approximate)

Need	Tool	Cost
Hovers, fades, marquees, button effects	CSS transitions and keyframes	0 KB
Scroll reveals, page transitions, shared-element moves	Motion with LazyMotion and m components	about 5 KB initially, features load on demand
Scroll-scrubbed hero, pinned sections	GSAP + ScrollTrigger, dynamic import on Home after first paint	about 40 KB, deferred
Smooth inertial scrolling	Lenis	about 3 KB
Avoid	Three.js, Lottie players, large particle libraries	60 KB and up each

Animation rules: animate only transform and opacity, keep durations between 200 and 600 ms, never delay content that is the largest paint, honour prefers-reduced-motion, and test on a mid-range Android phone rather than a laptop.

System architecture

One Vercel project serves the static React app from its CDN and runs FastAPI as a serverless function under /api. The browser talks to a single domain, so CORS never comes up, and the three outside services (Neon, Cloudinary, and Google Sheets) sit behind the API.
[embed: node/7dbb29b8-a811]
Visitors load pages from the CDN and read data through /api. Admins call the

same API with a login cookie. FastAPI only signs the photo upload; the photo itself travels from the browser to Cloudinary.

Design rules forced by the free tier

Constraint	Fact (checked against vendor docs)	Design response
Request body limit	Vercel Functions accept at most 4.5 MB per request or response	Photos never pass through FastAPI. The browser uploads straight to Cloudinary with a signed upload and the API stores only metadata.
Stateless functions	No persistent disk, cold starts, instances come and go	No local files and no in-memory sessions. Auth uses a signed cookie, and the database is the only state.
Database sleeps	Neon Free suspends compute after 5 minutes idle	The first request after a quiet spell is slower. Cache public reads at the CDN and use Neon's pooled connection string.
Database size	Neon Free gives 0.5 GB of storage per project	Store only text: events, form definitions, answers, messages. Photos live on Cloudinary.
Image credits	Cloudinary Free gives 25 credits a month, where one credit is 1,000 transformations, 1 GB of storage or 1 GB of bandwidth	Compress in the browser before upload and serve a few fixed sizes, so transformations stay low. Watch the usage dashboard monthly.

Function limits	Python bundles up to 500 MB uncompressed and up to 300 s duration on Hobby	Not a problem for this app. Keep the contact handler to a few seconds anyway.
Plan terms	Hobby is for non-commercial use	Confirm DevCatalyst qualifies before launch.

Caching rule: every public GET endpoint returns Cache-Control: public, s-maxage=60, stale-while-revalidate=300. Vercel's CDN then answers most visitors without waking the function or the database. Admin endpoints are never cached.

Data model

Neon Postgres holds seven tables. An event is the central object: it owns the public listing, the gallery album and the optional registration form, so "create an event, then add photos" is one workflow. Form fields live in a JSONB column and answers are keyed by field id, which is what makes the form builder dynamic without schema changes.

Table	Key columns	Notes
admins	id, email (unique), password_hash, role (owner or editor), is_active, last_login_at	Created by a seed script. No public sign-up.
events	id, slug (unique), title, summary, description (markdown), starts_at, ends_at, venue, cover_public_id, status (draft, published, archived)	Index on (status, starts_at). Past published events appear in the gallery.
event_photos	id, event_id (cascade), cloudinary_public_id, width, height, caption, sort_order	Metadata only. Width and height let the page reserve space and avoid layout shift.

forms	id, event_id, schema (JSONB), version, opens_at, closes_at, max_responses, confirmation_message, send_confirmation_email	One active form per event. Version increments on each schema change.
registrations	id, form_id, form_version, answers (JSONB), email_normalized, status (confirmed, waitlisted, cancelled), created_at	Unique on (form_id, email_normalized) to stop duplicates.
contact_messages	id, name, email, subject, message, status (new, read, archived), sheet_synced, email_sent, created_at	Database copy is the source of truth. The two flags drive retries.
login_attempts	id, email, ip_hash, succeeded, attempted_at	Rate limiting without Redis, since functions are stateless.

Form definition stored in forms.schema

```
{
  "fields": [
    {"id": "full_name", "type": "text", "label": "Full name", "required": true,
"max_length": 80},
    {"id": "email", "type": "email", "label": "Email", "required": true},
    {"id": "year", "type": "select", "label": "Year of study", "required": true,
"options": ["1st", "2nd", "3rd", "4th"]},
    {"id": "topics", "type": "checkbox_group", "label": "Topics of interest",
"options": ["Web", "AI", "Cloud"]},
    {"id": "agree", "type": "consent", "label": "I accept the code of conduct",
"required": true}
  ]
}
```

Supported field types in version 1: text, textarea, email, phone, number, date, select, radio, checkbox_group, consent. Each accepts required, a label and help text, plus type-specific rules such as max_length, min, max or options. File upload is phase 2.

Schema rules that protect existing data

- Answers are keyed by field id, never by label, so renaming a label does not orphan old answers.

- Each registration records the form_version it was submitted against, so exports can show the right columns for old rows.
- Once a form has registrations, admins can add fields and edit labels, but removing a field or changing its type archives it instead of deleting its answers.
- Exactly one field has type email. The API copies its lowercased value to email_normalized for duplicate checks and confirmation mail.

Backend design

The backend is a standard FastAPI app with SQLAlchemy 2 (async), Pydantic v2 and Alembic migrations. All routes live under /api, and a thin api/index.py exposes the app to Vercel. Keep the application code in a root-level app/ package, not inside api/, because every extra Python file in api/ would be deployed as its own function.

Repository layout

```
devcatalyst/
  vercel.json
  requirements.txt
  api/
    index.py          # from app.main import app
  app/
    main.py           # app factory, middleware, router registration
    core/             # settings, security, database, error handlers
    models/           # SQLAlchemy models
    schemas/          # Pydantic request and response models
    routers/          # auth, events, gallery, registrations, contact, admin_*
    services/         # dynamic_forms, mailer, sheets, cloudinary
  migrations/         # Alembic
  frontend/           # Vite + React app
```

vercel.json

```
{
  "buildCommand": "cd frontend && npm ci && npm run build",
  "outputDirectory": "frontend/dist",
  "rewrites": [
    { "source": "/api/(.*)", "destination": "/api/index" },
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

Vercel serves real static files first, so the last rule only catches client-side routes such as /events/hackathon. Routers are registered with the /api prefix so paths match what the browser sends.

Public endpoints (no login)

Method and path	Purpose
GET /api/events	Published events, filter by upcoming or past
GET /api/events/{slug}	One event, plus its form definition when registration is open
POST /api/events/{slug}/register	Submit a registration
GET /api/gallery	Albums: published events that have photos, with cover and count
GET /api/gallery/{slug}	Photos of one album, ordered
POST /api/contact	Submit a contact message
GET /api/health	Uptime check

Admin endpoints (login required)

Method and path	Purpose
POST /api/auth/login, POST /api/auth/logout, GET /api/auth/me	Session handling
POST /api/auth/change-password	Admin changes own password
GET, POST /api/admin/events; GET, PATCH, DELETE /api/admin/events/{id}	Event management, including draft and publish
PUT /api/admin/events/{id}/form	Save the form definition and open or close dates
GET /api/admin/events/{id}/registrations, .../registrations.csv	List with filters, and CSV export
PATCH, DELETE /api/admin/registrations/{id}	Change status or remove an entry
POST /api/admin/uploads/sign	Returns signed Cloudinary upload parameters
POST, PATCH, DELETE /api/admin/events/{id}/photos	Attach uploaded photos, reorder, remove
GET, PATCH /api/admin/messages	Read and triage contact messages
GET, POST, PATCH /api/admin/admins	Manage admin accounts (owner role only)

Authentication and abuse protection

- Passwords are hashed with Argon2id (argon2-cffi) with a 12-character minimum.
- A successful login sets a signed JWT in an HttpOnly, Secure, SameSite=Strict cookie that expires after 8 hours. JavaScript never sees the token, and the shared domain means no CORS configuration.

- State-changing admin requests must send Content-Type: application/json plus an X-Requested-With header, as a second CSRF barrier.
- Login is limited to 5 failures per 15 minutes per email and IP, counted in login_attempts.
- Public POST endpoints use a per-IP limit, a hidden honeypot field and a minimum fill time. Add Cloudflare Turnstile (free) on the contact and registration forms if spam appears.
- Roles are enforced in a single FastAPI dependency: owner can manage admins, editor can do everything else.

Dynamic form validation (services/dynamic_forms.py)

1. Load the form for the event and check it is open, inside its date window and under max_responses.
2. Build a Pydantic model at runtime from the schema with create_model, cached in memory by form id and version.
3. Validate the submitted answers. Unknown keys are rejected and values are trimmed and length-limited.
4. In one transaction, lock the form row, recheck capacity, insert the registration and rely on the unique index for duplicate emails.
5. Return 201, then trigger the confirmation email if enabled. A mail failure never undoes the registration.

Database access: use Neon's pooled connection string with SQLAlchemy's NullPool, so each invocation opens a short-lived connection instead of holding a pool that the next cold instance cannot reuse. Migrations run from your laptop or CI with alembic upgrade head, never inside a request.
Errors: every failure returns {"error": {"code": "...", "message": "...", "fields": {...}}} so the frontend can show field-level messages without special cases.

Feature implementation

Three features carry most of the complexity: the registration form builder, the gallery upload pipeline and the contact alert chain. Each is designed so the browser or the database does the heavy work, and the serverless function stays short-lived.

1. Registration form builder

- The admin builder has a field palette, drag-and-drop ordering (dnd-kit), a settings panel per field, and a live preview that uses the same renderer component as the public page, so what the admin sees is what visitors get.
- Form-level settings: open and close dates, maximum responses, confirmation message, and a switch for the confirmation email.

- A "copy form from another event" action saves time because most events reuse the same questions.
- The public renderer turns the stored schema into a React Hook Form setup with a generated Zod schema. Client checks give instant feedback, and the server repeats every check because it is the authority.
- The registrations screen has search, a status filter, inline status changes and a CSV export generated by the API. Columns follow the form version of each row.

2. Gallery and photo upload pipeline

1. The admin creates an event, then opens its Photos tab and drops in files.
2. The browser resizes each image to at most 2400 px on the long side and re-encodes it as WebP or JPEG at about 82% quality (browser-image-compression). This also strips GPS data from EXIF.
3. The browser asks POST /api/admin/uploads/sign for signed parameters. The Cloudinary API secret stays on the server.
4. The browser uploads files straight to Cloudinary, three at a time, with a progress bar per file, and receives each file's public_id, width and height.
5. The browser posts the list to POST /api/admin/events/{id}/photos, and the API stores the metadata with a sort order.
6. The public gallery builds URLs with Cloudinary transformations f_auto,q_auto at three fixed widths (400, 800, 1600 px) and uses srcset, width and height attributes, loading="lazy" and a tiny blurred placeholder. The full-screen viewer loads the 1600 px version only when opened.
7. Deleting a photo makes the API remove it from Cloudinary first, then delete the row.

Using only three fixed widths keeps Cloudinary's transformation count predictable, because each size is generated once and then served from cache.

3. Contact form, Google Sheet and email alerts

1. The API validates the message: length caps, honeypot, per-IP rate limit and stripped line breaks in the name and subject to prevent email header injection.
2. It inserts the row into contact_messages first, so nothing is lost if a later step fails.
3. It then appends the row to the Google Sheet with a 5-second timeout, before answering the visitor. Serverless functions can be frozen once the response is sent, so background tasks after the response are not reliable.
4. **Sheet append:** a Google service account (credentials stored as a base64 environment variable) appends a row with timestamp, name,

email, subject and message. Share the sheet with the service account's email address as Editor. The sync Google client runs in a worker thread with asyncio.to_thread.

5. **Email alert from the Sheet:** a Google Apps Script bound to the contact sheet emails the team about every row whose Alerted column is empty, then stamps that column. It uses MailApp, so the mail leaves from the Google account that owns the script, with Reply-To set to the visitor. The API holds no email password. The script and trigger setup are below.
6. The API sets sheet_synced on the row and returns 201 even if the Sheet step failed. The visitor sees a success state either way.
7. A daily Vercel Cron job calls a secret-protected endpoint that retries every row where sheet_synced is still false. Hobby plans limit cron frequency, so check the current limit before relying on a shorter interval.

Apps Script alert (bound to the sheet)
Open the sheet, choose Extensions, then Apps Script, and paste the script below. The sheet needs a tab named Contact with the header row Timestamp, Name, Email, Subject, Message, Alerted. The API fills the first five columns and the script fills the last.

```
const ALERT_TO = 'lead@example.com,team@example.com'; // your team addresses
const SHEET_NAME = 'Contact';

function sendNewMessageAlerts() {
  const lock = LockService.getScriptLock();
  lock.waitLock(20000);
  try {
    const sheet = SpreadsheetApp.getActive().getSheetByName(SHEET_NAME);
    const last = sheet.getLastRow();
    if (last < 2) return;
    const rows = sheet.getRange(2, 1, last - 1, 6).getValues();
    const clean = s => String(s).replace(/[\r\n]+/g, ' ').slice(0, 120);
    rows.forEach((r, i) => {
      if (r[5]) return; // already alerted
      try {
        MailApp.sendEmail({
          to: ALERT_TO,
          replyTo: clean(r[2]),
          subject: '[DevCatalyst] New message: ' + clean(r[3]),
          body: 'From: ' + clean(r[1]) + ' <' + clean(r[2]) + '>\n\n' + r[4]
        });
        sheet.getRange(i + 2, 6).setValue(new Date());
      } catch (err) {
        sheet.getRange(i + 2, 6).setValue('ERROR: ' + err.message);
```

```
      }
    });
  } finally {
    lock.releaseLock();
  }
}
```

Trigger setup

1. In Apps Script, open Triggers, add a trigger for sendNewMessageAlerts, event source Time-driven, every minute. Approve the permissions once.
2. Optionally add a second trigger, event source From spreadsheet, event type On change, for near-instant alerts. Test it by submitting a real message, because change triggers do not always fire for rows added through the Sheets API.
3. The Alerted stamp makes the script safe to run repeatedly. A missed trigger only delays an alert, and nothing is lost or sent twice. To resend one, clear its Alerted cell.

Limits to know: MailApp has a daily recipient quota (roughly 100 on a free Google account and 1,500 on Workspace, approximate) and time-driven triggers have a daily runtime cap. A club contact form stays far below both, but check Google's Apps Script quotas page before launch.

Open decision, registration confirmation emails: this script only covers contact messages. For participant confirmations, either the API calls a second Apps Script web app that sends from the same account, or the form shows an on-screen confirmation only. Until you choose, the form shows an on-screen confirmation.

Frontend design

The public site is a prerendered, code-split React app where animation is layered on top of fast, plain content. Visitors never download admin code, and every animation library beyond CSS and a 5 KB Motion core loads after the first paint.

Routes

Route	Page	Loaded
/	Home	Main bundle
/about	About us	Lazy chunk
/events, /events/:slug	Events list, event detail with registration form	Lazy chunks
/gallery, /gallery/:slug	Album list, album with viewer	Lazy chunks

/contact	Contact	Lazy chunk
/admin/*	Admin panel	Separate lazy chunk, never fetched by visitors

Source layout (frontend/src)

```
main.tsx
app/            routes, providers, layout (Navbar, Footer, PageTransition)
pages/          Home, About, Events, EventDetail, Gallery, Album, Contact, NotFound
features/
  registration/ DynamicForm, field renderers, buildZod.ts
  gallery/      MasonryGrid, Lightbox, cloudinary.ts (URL builder)
  events/       EventCard, queries
  contact/      ContactForm
admin/          lazy: auth, events, form-builder, registrations, photos, messages, admins
components/ui/  Button, Input, Select, Modal, Toast
lib/            api client (generated types), motion presets, seo helpers
styles/         globals.css with design tokens
```

Animation plan by page

Page	Signature effects	Tool
Home	Split-text headline reveal, animated gradient background, staggered event cards, optional pinned scroll story	CSS, Motion, GSAP (deferred)
About	Timeline line that draws as you scroll, card hover tilt	CSS scroll-driven animation with a Motion fallback
Events	Animated filter tabs, card lift on hover, form fields with focus micro-interactions	Motion layout, CSS
Gallery	Masonry fade-in on view, cover-to-album shared transition, swipeable lightbox	Motion layoutId and drag
Contact	Floating labels, animated checkmark on success	CSS, inline SVG

Route changes	Short fade and slide between pages	Motion AnimatePresence

Performance budget (targets enforced in CI with size-limit)

Metric	Target
Initial JavaScript on public pages	120 KB gzipped or less
Admin chunk	Under 250 KB gzipped, fetched only after admin login
CSS	Under 20 KB gzipped
Largest Contentful Paint	2.0 s or less on a mid-range phone over 4G
Cumulative Layout Shift	0.05 or less
Interaction to Next Paint	200 ms or less
Fonts	Two files at most, Latin subset
Lighthouse mobile performance	90 or higher

Techniques that keep it light

- Route-level React.lazy, plus Rollup manual chunks for React and for Motion.
- GSAP and Lenis are dynamically imported inside requestIdleCallback on Home only.
- No heavy utility libraries: native Intl for dates, per-icon imports for Lucide, no UI kit on public pages.
- The hero image is the only fetchpriority="high" asset, served as WebP or AVIF under about 120 KB, with a preconnect to the Cloudinary host.
- Every image has explicit width and height, and skeletons match the final layout, so nothing jumps.
- TanStack Query uses a 60-second staleTime and prefetches an event's data when a card is hovered.
- React 19's built-in <title> and <meta> support handles per-page SEO without a helmet library.
- Hashed asset filenames get long-lived caching and Brotli from Vercel automatically.

Accessibility and motion safety: every animation checks prefers-reduced-motion and falls back to an instant state. Focus rings stay visible, the lightbox traps focus and closes on Escape, and form errors are announced to screen readers.

Sharing previews (phase 2): a single-page app cannot give each event its own link preview on WhatsApp or LinkedIn. A Vercel rewrite for crawler user agents can send /events/:slug to a small FastAPI route that returns minimal HTML with Open Graph tags.

Admin panel

The admin panel lives at /admin inside the same React app but in its own lazy chunk, so it can use heavier components (drag-and-drop, data tables) without affecting visitors. Everything an organiser does centres on an event.

Screens

Screen	What the admin does
Login	Signs in with email and password. Repeated failures show a lockout message.
Dashboard	Sees upcoming events, registrations this week, unread messages and the latest sign-ups.
Events	Lists events by status. Creates or edits title, slug, date and time, venue, markdown description, cover image and status.
Event, Form tab	Builds the registration form, sets open and close dates and capacity, previews it.
Event, Registrations tab	Searches and filters entries, changes status, exports CSV.
Event, Photos tab	Drops in photos, reorders, edits captions, picks the cover, deletes.
Messages	Reads contact messages, marks them new, read or archived, replies by email.
Admins (owner only)	Adds, disables or resets other admins.
Account	Changes own password.

Core workflows

1. **Run a new event:** Events, New, fill the details as a draft. Open the Form tab, build the form and set dates and capacity. Preview it, then publish. Share the /events/{slug} link. After registration closes, export the CSV.
2. **Publish a gallery:** open the finished event, drop photos into the Photos tab, reorder them and choose a cover. The event now appears in the Gallery as an album with no extra step.
3. **Handle a contact message:** the alert email arrives, open Messages,

mark it read and reply from your mail client.

Security measures

- /admin is disallowed in robots.txt and carries noindex. The interactive API docs at /api/docs are switched off in production.
- vercel.json sets response headers: a Content Security Policy that allows only your own origin plus the Cloudinary hosts, X-Content-Type-Options: nosniff and a strict Referrer-Policy.
- All secrets (database URL, JWT secret, Cloudinary secret, Google credentials, cron secret) live in Vercel environment variables and never in the repository.
- Admin accounts are created by a seed script run by an owner. Disable an account instead of deleting it when someone leaves the team.
- Registrations contain personal data. Collect only what each event needs, show a short privacy notice on the form, and provide a way to delete a participant's data on request. Check the obligations under India's Digital Personal Data Protection Act for your situation.

Deployment and operations

A single GitHub repository connects to one Vercel project. Every pull request gets a preview URL, and merging to maindeploys production, with database migrations run first.

Environments

Environment	How it runs	Database
Local	uvicorn app.main:app --reload plus npm run dev, with the Vite dev server proxying /api to 127.0.0.1:8000	A separate Neon project for development
Preview	Automatic Vercel deployment per pull request	The development database
Production	Vercel deployment from main on your custom domain, HTTPS issued automatically	The production Neon project

Environment variables (set in Vercel, never committed)

Variable	Used for

DATABASE_URL	Neon pooled connection string
JWT_SECRET	Signing the admin session cookie
CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET	Signing uploads and deleting photos
GOOGLE_SERVICE_ACCOUNT_B64, CONTACT_SHEET_ID	Appending contact rows to the sheet. The alert email address list lives in the Apps Script, not here
CRON_SECRET	Protecting the retry endpoint
TURNSTILE_SECRET (optional)	Bot check on public forms
VITE_CLOUDINARY_CLOUD_NAME, VITE_TURNSTILE_SITE_KEY	Public values exposed to the frontend build

CI/CD

1. On every pull request, GitHub Actions runs frontend lint, type check and the size-limit bundle check, plus backend ruff and pytest against a throwaway Postgres service.
2. On merge to main, a job runs alembic upgrade head against production first. Migrations are additive and backward compatible, so the old code keeps working while the new deployment rolls out.
3. Vercel builds and publishes the frontend and the function.
4. A weekly scheduled GitHub Actions job runs pg_dump and stores the file as a private artifact. Neon's free plan keeps only a short restore window, so this is your real backup.

Monitoring

- Vercel function logs for errors, and Vercel's built-in analytics and Speed Insights for real-user Core Web Vitals.
- An uptime monitor such as UptimeRobot pings /api/health. That endpoint deliberately skips the database, because pinging the database every few minutes would keep Neon awake and use up its monthly compute hours.
- Optional: Sentry's free tier for backend and frontend exceptions.

Free-tier watchlist

Service	Limit to watch	If you get close
Neon Free	100 compute hours and 0.5 GB storage per project each month	Lean on CDN caching, trim old data, or move to the pay-as-you-go Launch plan

Cloudinary Free	25 credits a month across transformations, storage and bandwidth	Keep to three image sizes, compress before upload, or upgrade
Vercel Hobby	Usage limits in the dashboard, non-commercial terms	Check monthly, move to Pro if the club turns commercial
Apps Script email quota	Daily recipient and runtime quotas for the account that owns the script	Alert on fewer messages, use a Workspace account, or add a Resend sender to the API

Roadmap and risks

Build in five phases, about eight weeks for two or three people working part time. Each phase ends at a gate you can test on a live preview, so problems with Vercel limits, email or uploads surface early rather than at launch.

[embed: node/42201ef4-c9fe]

Main risks and how the design handles them

Risk	Likelihood	Mitigation
Cold starts slow the first visit	Medium	Cache public reads at the CDN, keep the function bundle small, import heavy libraries lazily, measure in preview
Neon wakes slowly after idle	Medium	Same CDN caching. Do not ping the database from monitors
A large gallery uses up Cloudinary credits	Medium	Three fixed sizes, browser compression, monthly check. The URL builder is one file, so moving to another image host is a small change

Apps Script alerts stop or hit a quota	Low	The Alerted column and the admin Messages screen show unsent rows. Messages are always saved in the database first
Spam on public forms	Medium	Honeypot, rate limits, optional Turnstile
Animations hurt low-end phones	Medium	Transform and opacity only, reduced-motion support, bundle budget in CI, test on a mid-range Android
Hobby plan terms do not fit the club	Low	Confirm non-commercial use now. Fallback is Vercel Pro or another static host plus a separate API host

Sources

- Vercel Functions limits: 4.5 MB body limit, Python bundle size, Hobby duration.
- Neon Free plan limits: 0.5 GB storage, 100 compute hours, scale to zero after 5 minutes.
- Neon plans: plan comparison and restore window.
- How Cloudinary credits work: what one credit buys.

Library bundle sizes quoted in this document are approximate and should be confirmed with a bundle analyzer once the app exists.